



Planteamiento inicial · App gestor de tareas

Documento base del proyecto

Aplicación full stack de gestión de tareas con usuarios, autenticación, avatar y panel personal. El nombre de la aplicación es **Dolt App**.

Enfoque general

La aplicación permitirá que cada usuario tenga su propia cuenta y gestione únicamente sus tareas.

Las funcionalidades principales serán:

- registro e inicio de sesión
- avatar de usuario (con placeholder por defecto desde Cloudinary)
- creación, edición y borrado de tareas
- filtrado por fecha o estado
- ordenación por fecha o completado
- edición del perfil

Mapa del proyecto

Backend

- API REST con usuarios y tareas
- autenticación con JWT
- contraseñas cifradas con bcrypt
- avatar almacenado con Cloudinary (carpeta `doitusers`)
- avatar con URL de placeholder por defecto

- **eliminación del avatar antiguo de Cloudinary** al actualizar el avatar
- manejo centralizado de errores con `utils/errorHandler.js` y `utils/notFoundHandler.js`
- relación entre usuarios y tareas mediante MongoDB

Frontend

- aplicación en React con Vite (nombre del proyecto: `doitapp`)
 - navegación con React Router
 - dashboard para gestionar tareas
 - portal de usuario para editar perfil
-

▼ Backend

Estructura general

La organización del backend será modular y clara:

- `config`
- `models`
- `controllers`
- `routes`
- `middlewares`
- `utils`
- `index.js`
- `.env`

Colecciones de MongoDB

Tendremos dos colecciones principales.

Users

Campos principales:

- `username`
- `email`
- `password`
- `avatar`

`avatar` será un **string** con la URL final devuelta por Cloudinary. Si el usuario no sube foto al registrarse, se asignará automáticamente una URL de avatar placeholder alojada en Cloudinary.

Tasks

Campos principales:

- `title`
- `date`
- `status`

- `user`

Relación entre colecciones

- un usuario puede tener muchas tareas
- cada tarea pertenece a un único usuario

La relación se resolverá desde la colección **Tasks**, guardando en cada tarea el campo `user` con la referencia al propietario.

No es necesario que el modelo de **Users** tenga un array de ids de tareas, porque eso duplicaría información y complicaría las actualizaciones.

Controladores

Auth y usuario

- `register`
- `login`
- `getMe`
- `updateUsername`
- `updatePassword`
- `updateAvatar`

Tareas

- `getTasks`
- `createTask`
- `updateTask`
- `deleteTask`

Rutas principales

Auth y usuario

- registro
- login
- perfil

- edición de nombre
- edición de contraseña
- edición de avatar

Tareas

- obtener tareas
- crear tarea
- editar tarea
- borrar tarea

Middlewares necesarios

- autenticación con JWT
- subida de archivos para avatar
- manejo de errores
- ruta no encontrada

Cloudinary

Cloudinary se utilizará para la gestión del avatar del usuario.

Necesitaremos:

- subir imagen a la carpeta `doitusers`
- guardar la URL final en MongoDB
- **eliminar la imagen anterior** de Cloudinary cuando el usuario cambie su avatar (para no acumular imágenes huérfanas)

Reglas importantes

- la contraseña se almacenará cifrada con bcrypt
- el login devolverá un token JWT
- cada usuario solo podrá ver y modificar sus propias tareas
- los errores de validación devolverán la clave `error` (no `message`)
- los errores de autenticación y de token devolverán la clave `message`



▼ Frontend

Tecnologías

- Vite
- React 19
- React Router 7
- react-hook-form → gestión de formularios sin useState manual
- Tailwind CSS v4 → estilos con tema custom dark/orange
- useState, useEffect, useCallback, memo
- fetch

Páginas principales

- `Home`
- `Login`
- `Register`
- `Dashboard`
- `Profile`
- `NotFound`

Componentes principales

Tareas

- `TaskForm` (con react-hook-form)
- `TaskList`
- `TaskCard`
- `TaskFilters`

Usuario

- `UsernameForm` (con react-hook-form)
- `PasswordForm` (con react-hook-form + confirmación de contraseña)

- `AvatarForm` (con `react-hook-form`)

Estructura general

- `Navbar` (con `NavLink` y estilos Tailwind; sin CSS separado)
- `ProtectedRoute`

Funcionalidad del frontend

Auth

- registrar usuario
- iniciar sesión
- guardar token
- proteger rutas privadas

Tareas

- listar tareas del usuario (respuesta directa como array, sin clave `tasks`)
- crear tarea
- editar tarea
- borrar tarea
- filtrar por fecha o estado
- ordenar por fecha o completado

Perfil

- cambiar avatar (recarga usuario con `loadUser`)
- cambiar nombre de usuario (recarga usuario con `loadUser`)
- cambiar contraseña con confirmación client-side

Contexto de autenticación

El contexto expone `loadUser` además de `token` , `user` , `setUser` , `login` y `logout` . Esto permite que los formularios del perfil recarguen el usuario desde el backend sin tener que actualizar el estado manualmente.

Los errores del backend se detectan comprobando `res.error` (no `res.message`).

Comunicación con el backend

La URL del backend se lee desde `import.meta.env.VITE_API_URL`. En local apunta a `http://localhost:3000/api`. En producción (Render) se configura en las variables de entorno de Vercel.

MVP

La aplicación final deberá permitir:

- autenticación de usuarios
- gestión individual de tareas por usuario
- edición del perfil
- subida de avatar con Cloudinary (con placeholder si no se sube foto)
- organización clara entre backend y frontend

Librerías necesarias

▼ Backend

- `express` → para crear la API y definir rutas
- `mongoose` → para conectar con MongoDB y definir modelos
- `dotenv` → para leer variables de entorno desde `.env`
- `cors` → para permitir la comunicación entre frontend y backend
- `bcrypt` → para cifrar contraseñas
- `jsonwebtoken` → para generar y validar tokens de autenticación
- `cloudinary` → para subir, gestionar y **eliminar** el avatar del usuario
- `multer` → para recibir archivos enviados desde formularios
- `multer-storage-cloudinary` → para conectar la subida de archivos con Cloudinary

▼ Frontend

- `react-router-dom` → para crear la navegación entre páginas
- `react-hook-form` → para gestionar formularios sin useState por campo
- `tailwindcss` + `@tailwindcss/vite` → para los estilos con tema custom
- `vite` → para levantar y construir el proyecto frontend